

(Common)① Insertion Sort :

Insertion sort works the same way many people sort a hand of playing cards. We start with an empty left hand and the cards face down on the table. We then remove one card at a time from the table and insert it into the correct position in the left hand. To find the correct position for a card, we compare it with each of the cards already in the hand, from right to left. At all times, the cards held in the left hand are sorted, and those cards were originally the top cards of the pile on the table.

→ Input : a sequence of numbers $\{a_1, \dots, a_n\}$ (array $A[1..n]$)

Output : a permutation $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the I/P sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

The I/P numbers are sorted in place, in a non-decreasing order.

Pseudocode :-

INSERTION-SORT (A)

1 for $j \leftarrow 2$ to $\text{length}[A]$

2 do $\text{Key} \leftarrow A[j]$

3 ▷ Insert $A[j]$ into the sorted sequence $A[1..j-1]$

4 $i \leftarrow j-1$

5 while $(i > 0 \text{ and } A[i] > \text{Key})$

6 do $A[i+1] \leftarrow A[i]$

7 $i \leftarrow i-1$

8 $A[i+1] \leftarrow \text{Key}$

The symbol $\triangleright$ is used to indicate a comment. The numbers that we wish to sort are also known as the Keys.

→ Loop invariant: At the start of each iteration of the ② for loop of lines 1-8, the subarray $A[1..j-1]$ consists of the elements originally in $A[1..j-1]$ but in sorted order.

We use loop invariants to help us understand why an algorithm is correct. We must show 3 things about a loop invariant:-

- (i) Initialization: It is true prior to the 1st iteration of the loop.
- (ii) Maintenance: If it is true before an iteration of the loop, it remains true before the next iteration (i.e. at the end of the current iteration).
- (iii) Termination: When the loop terminates, the invariant gives us a useful property that helps show that the algorithm is correct.

② Analyzing algorithms

→ It means predicting the resources that an algorithm requires. Usually, it is just the computational time that we want to measure.

→ The "running time" of an algorithm on a particular VP is the no. of primitive operations or steps executed.

→ For a sorting algorithm the best case occurs when the I/P array is already sorted. The worst case occurs when the I/P array is reverse sorted.

→ We are usually interested in the worst-case running time of an algorithm because: (i) the algorithm will never take any longer (ii) the average case is often roughly as bad as the worst case, (iii) for some algorithms, the worst case occurs fairly often.

→ Order of growth

We are interested in the "rate of growth" or "order of growth" of the running time. Hence, if the running time is of the form $an^2 + bn + c$, we consider only the leading terms (i.e. an^2).

because for large input size n , the lower order terms are ③ relatively insignificant. We also ignore the constant coefficient of the leading term, since they are less significant than the rate of growth.

→ The worst-case running time ($an^2 + bn + c$) of insertion sort is thus said to be $\Theta(n^2)$.

Exercise 2.2-3: Selection Sort

You sort n numbers of array A by first finding the smallest element of A and exchanging it with the element in $A[1]$. Then find the second smallest element of A and exchange it with $A[2]$. Continue in this manner for the first $n-1$ elements of A .

③ Designing algorithms

We have seen the "incremental approach" to designing algorithms in insertion sort: having sorted the subarray $A[1..j-1]$, we insert the element $A[j]$ into its proper place to obtain the sorted subarray $A[1..j]$.

→ Divide-and-conquer approach

Divide-and-conquer algorithms are recursive in nature and involve the following steps:-

- i) Divide the problem into a number of subproblems
- ii) Conquer the subproblems by solving them recursively. If the subproblem sizes are small enough, however, just solve the subproblems in a straightforward manner.
- iii) Combine the solutions to the subproblems into the solution for the original problem.

The merge sort algorithm follows the divide-and-conquer paradigm and operates as follows:-

- i) Divide: Divide the n -element sequence to be sorted into

Two subsequences of $n/2$ elements each.

- (ii) Conquer: Sort the two subsequences recursively using mergesort.
- (iii) Combine: Merge the two sorted subsequences to produce the sorted answer.

The recursion bottoms out when the sequence to be sorted has length 1.

→ The MERGE-SORT algorithm uses an auxiliary auxiliary procedure $MERGE(A, p, q, r)$ where p, q, r are indices of the array A such that $p \leq q < r$. It assumes that the ^{sub}arrays $A[p..q]$ and $A[q+1..r]$ are in sorted order and merges them to produce a single sorted subarray $A[p..r]$.

→ Suppose we have two piles of card face up on a table. Each pile is sorted, with the smallest cards on top. We wish to merge the two piles into a single o/p pile, which is to be face down on the table. Our basic step consists of choosing the smaller of the two cards on top of the face-up piles, removing it from its pile, and placing this card face down on the o/p pile. We repeat this until one i/p pile is empty, at which time we just take the remaining i/p pile and place it face down onto the o/p pile. Since we perform at most n basic steps, where $n = r - p + 1$, merging takes $\Theta(n)$ time.

→

$MERGE(A, p, q, r)$

1 $n_1 \leftarrow q - p + 1$

2 $n_2 \leftarrow r - q$

3 create arrays $L[1..n_1+1]$ and $R[1..n_2+1]$

4 for $i \leftarrow 1$ to n_1

5 do $L[i] \leftarrow A[p+i-1]$

6 for $j \leftarrow 1$ to n_2

7 do $R[j] \leftarrow A[q+j]$

8 $L[n_1+1] \leftarrow \infty$ $\triangleright$ Put a sentinel value at end which would be greater than all values in R (5)

9 $R[n_2+1] \leftarrow \infty$ $\triangleright$ sentinel value

10 $i \leftarrow 1$

11 $j \leftarrow 1$

12 for $k \leftarrow p$ to r

13 do if $L[i] \leq R[j]$

14 then $A[k] \leftarrow L[i]$

15 $i \leftarrow i+1$

16 else $A[k] \leftarrow R[j]$

17 $j \leftarrow j+1$

$\rightarrow$ MERGE-SORT (A, p, r)

1 if $p < r$

2 then $q \leftarrow \lfloor (p+r)/2 \rfloor$

3 MERGE-SORT(A, p, q)

4 MERGE-SORT($A, q+1, r$)

5 MERGE(A, p, q, r)

$\rightarrow$ To sort $A[1..n]$ we make a call MERGE-SORT($A, 1, n$)

$\rightarrow$ The merge sort algorithm is $\Theta(n \lg n)$.

Facts related to growth of functions:—

(i) Exponential functions grow faster than polynomial functions which grow faster than polylogarithmic functions (i.e. logarithmic functions of some degree).

(ii) The base of a logarithm does not matter asymptotically, but the base of an exponential and the degree of a polynomial do.

Stirling's approximation

⑥

$$\begin{aligned}\lg L_n &= \lg(n \cdot n-1 \cdot n-2 \cdot \dots \cdot 1) \\&= \lg n + \lg(n-1) + \lg(n-2) + \dots + 1 \\&= \sum_{k=1}^n \lg k \\&\approx \int_1^n \lg x \, dx \\&= [x \lg x - x]_1^n \\&= n \lg n - n + 1\end{aligned}$$

$$\text{or, } \lg L_n = \Theta(n \lg n)$$

→ Stirling's approximation allows you to compare factorial functions with polynomials or polylogarithmic functions.

$$\text{eg: } \lg n! = \Omega(2^{\lg n})$$

$$2^{\lg n} = n^{\lg 2} = n \quad [\because a^{\lg b} = b^{\lg a}]$$

$$\text{and } \lg n! = \Theta(n \lg n)$$

$$\text{Since, } n \lg n \text{ is } \Omega(n) \Rightarrow \lg n! = 2^{\lg n}$$

for more info. on comparison of rate of growth see
Cormen problem 3-3.